

Rock, Paper, Scissors

Create an application that simulates the classic game of "Rock, Paper, Scissors."

Screen Implementation

You will need to decide how to implement the following four screens. One must use Storyboards, another one XIBs, the third one should be done programmatically, and the last one is up to you. It's necessary to analyze the requirements of each screen and determine which approach best fits each one.

Requirements

Start Screen

Include a *LaunchScreen* with an image related to the game.

Configuration Screen

On the configuration screen, users will be able to customize their gaming experience, including the following elements and functions:

- Player's Name: Provide a text field where the player must enter their name. This is mandatory to proceed and play.
- Game Options: Users can choose between two game options: by rounds or by points.
 - For rounds, include a slider configured with values from 1 to 5, allowing players to choose the number of rounds needed to win.
 - For points, present three text fields: one for the win value, one for the lose value, and one for the required score to win the session.
- Win Value: The score the player will receive when winning a round.
- Lose Value: The score subtracted from the player when losing a round.
- Required Score to Win: The score the player needs to achieve to win the session.
- "Continue" Button: Present a "Continue" button that allows players to advance to the game after entering a valid name and configuring the winning rounds or points, depending on the case. The button remains hidden until all fields are correctly filled out.
- "Information" Button: Add an "Information" button that, when pressed, takes the user to another screen explaining the rules of the "Rock, Paper, Scissors" game. Include a section for acknowledgments, which will contain your name.

Game Screen

The main screen displays three buttons, each with an emoji hand symbol representing the "Rock," "Paper," or "Scissors" choices. The background of this screen should be gray. When the user selects an option, the application randomly selects the opponent's choice and displays it on the screen.

Determine the game's outcome by comparing the user's and opponent's selections and display whether the user won, lost, or tied.

- If the user wins, change the background of the screen to green to indicate victory.
- If the user loses, change the background to red to indicate defeat.
- In case of a tie, set the background to brown.

In all cases, a label should indicate the result with a string and always mention the player's name.

Game Mode

- If the user selects "by points," the application displays a point counter that starts at zero at the beginning of the game. When the user wins or loses a round, the points they proposed are awarded or deducted and added to the counter. The points update in real-time and are displayed on the screen for the user to track their progress. When the user reaches the proposed winning score, a victory alert is displayed. Accepting it removes the alert. Negative scores are not allowed.
- If the user chooses "by rounds," the application displays a round counter that starts at zero at the beginning of the game. Each won round adds 1 point to this counter. When the user reaches the number of rounds selected on the slider, a victory alert is displayed.

Next Turn:

After a round is played, a "Next Turn" button appears, allowing the user to proceed to the next round.

Game Reset:

Provide a "Reset Game" button that restores all options and values to their initial states.

History

The screen should have a button to view the game's history, which will be displayed on another screen. The game information will be stored in an array and shown in a TextView. Note that this information is available only while the application is running, and no data is stored in memory.